Verification and Validation Report: Crazy 1-0s Game

Team #25, The Crazy Four
Ruida Chen
Ammar Sharbat
Alvin Qian
Jiaming Li

March 8, 2026

1 Revision History

Date	Version	Notes
2026-03-01	1.0	Initial draft; defined test cases and verification methods.
2026-03-05	1.1	Updated with results from Rev 0 demo and automated test metrics.
2026-03-08	1.2	Finalized NFR evaluation, documented testing-driven changes, and replaced placeholder CI/coverage text with verified Rev 0 results.

2 Symbols, Abbreviations and Acronyms

Symbol	Description
FR	Functional Requirement
NFR	Nonfunctional Requirement
CI/CD	Continuous Integration / Continuous Deployment
API	Application Programming Interface
UI	User Interface
Dozenal	Base-12 number system
WS	WebSocket communication protocol

3 Functional Requirements Evaluation

Note: While the original VnV Plan specified automated Node console harnesses for Functional Requirements testing (e.g., ST-GM-01), the final evaluation was conducted via ***Manual UI-based Testing*** *on the deployed application. This transition was made to better validate the end-to-end integration of the frontend, backend, and real-time WebSocket engine under realistic network conditions.*

3.1 Game Manager: Start, Turn, Rules, Specials, End

Start new game initializes state (ST-GM-01)

- **Test Type:** Manual UI
- **Expected Output:** Player A finds the game room created by Player B in the game room lists, click join the game then UI transitions to the Game Screen, both avatars show 7 cards in hand, discard pile shows 1 top card, and UI visually indicates it is User A's turn.
- **Actual Output:** The actual results match the expected results.
- **Result:** PASS

Legal move: same suit (ST-GM-02)

- **Test Type:** Manual UI
- **Expected Output:** Card of the same suit moves to discard pile and turn indicator shifts to User B.
- **Actual Output:** The actual results match the expected results.
- **Result:** PASS

Legal move: sum equals 12(in decimal) base-12 (ST-GM-03)

- **Test Type:** Manual UI
- **Expected Output:** Card whose decimal value adds up to 12 with the top card is accepted and turn advances.

- **Actual Output:** The actual results match the expected results.
- **Result:** PASS

Special card: wild "10" suit selection (ST-GM-04)

- **Test Type:** Manual UI
- **Expected Output:** Selecting a wildcard prompts for a suit selection. UI updates to show forced suit, restricting next player.
- **Actual Output:** The actual results match the expected results.
- **Result:** PASS

Illegal move is rejected with guidance (ST-GM-05)

- **Test Type:** Manual UI
- **Expected Output:** Illegal cards are not highlighted and cannot be selected or played.
- **Actual Output:** The actual results match the expected results. Error message can be found in collapsible activity logs.
- **Result:** PASS

End of game detection (ST-GM-06)

- **Test Type:** Manual UI
- **Expected Output:** Successfully playing final card transitions UI to a summary screen declaring the winner of this round and displaying total scores.
- **Actual Output:** The actual results match the expected results.
- **Result:** PASS

3.2 Score Features: Dual-base, Highlights

Score calculated and shown in decimal or Dozenal in corresponding mode(ST-SC-01)

- **Test Type:** Manual UI
- **Expected Output:** Round Result popup correctly summarizes lost hand points in Decimal or Dozenal (e.g., 10_{12}) notations correctly.
- **Actual Output:** The The actual results match the expected results.
- **Result:** PASS

Valid moves highlighted exactly (ST-SC-03)

- **Test Type:** Manual UI
- **Expected Output:** Exactly the cards that are legal to play are visually highlighted/raised, while others are dimmed/disabled. This is not applied to sum-to-10 rule.
- **Actual Output:** The actual results match the expected results. Wild cards are hightlighted with special icons.
- **Result:** PASS

3.3 Login Manager: Account, Sessions, Guest, Validation

Account creation creates unique user and session (ST-LM-01)

- **Test Type:** Manual UI
- **Expected Output:** Valid registration logs user in. Duplicate registration is rejected with "username taken" error.
- **Actual Output:** The actual results match the expected results.
- **Result:** PASS

Login and logout preserve progress (ST-LM-02)

- **Test Type:** Manual UI
- **Expected Output:** Logging out and logging back in retains exact match history in Profile page.
- **Actual Output:** The actual results match the expected results.
- **Result:** PASS

Guest mode creates temporary session (ST-LM-03)

- **Test Type:** Manual UI
- **Expected Output:** "Play as Guest" creates temporary profile. Logging out deletes profile, preventing future logins to that guest account.
- **Actual Output:** The actual results match the expected results.
- **Result:** PASS

Credential validation gates access (ST-LM-04)

- **Test Type:** Manual UI
- **Expected Output:** Incorrect password rejected with "invalid credentials". Correct password proceeds to Lobby.
- **Actual Output:** The actual results match the expected results.
- **Result:** PASS

3.4 Data Manager: Storage, Retrieval, Update, Deletion

User data is stored on events that require persistence (ST-DM-01)

- **Test Type:** Manual UI
- **Expected Output:** Completing a game adds a correct entry to the Profile Match History.

- **Actual Output:** The actual results match the expected results.
- **Result:** PASS

Stored data can be retrieved on login or profile view (ST-DM-02)

- **Test Type:** Manual UI
- **Expected Output:** Profile page successfully fetches and renders stats (win rate) and latest 5 games' match history with scoring details.
- **Actual Output:** The actual results match the expected results.
- **Result:** PASS

Data updates after games and profile changes (ST-DM-03)

- **Requirement:** FR-16
- **Test Type:** Manual UI
- **Expected Output:** Changing Display Name in settings persists across full page refreshes.
- **Actual Output:** The actual results match the expected results.
- **Result:** PASS

User can permanently delete data (ST-DM-04)

- **Requirement:** FR-17
- **Test Type:** Manual UI
- **Expected Output:** Clicking "Delete Account" forces logout and permanently prevents logging back in with old credentials.
- **Actual Output:** The actual results match the expected results.
- **Result:** PASS

4 Nonfunctional Requirements Evaluation

4.1 Usability

Usability was evaluated through two-player manual play sessions in separate browser windows, followed by feedback from peers and instructor demo sessions. Players were able to understand the core card-play flow, read the score summary, and complete a match without needing rule explanations from the developers. The main usability issues that surfaced were the lobby create/join flow and the interaction design for arithmetic prompts, both of which are recorded in Section 7 as changes driven by testing feedback.

Result: PASS. Core gameplay was understandable and learnable, with minor UI revisions identified for polishing.

4.2 Performance

Performance was evaluated during local end-to-end testing of the frontend, backend, and WebSocket layer. Card plays, draws, turn updates, and score changes propagated quickly enough that players did not observe noticeable lag during normal play. Because the rule engine is implemented in shared TypeScript logic and invoked server-side, arithmetic/state transitions are lightweight relative to network overhead.

Result: PASS. In local testing the system remained responsive and gameplay updates were effectively immediate for Rev 0 usage.

4.3 Reliability

Reliability was evaluated through repeated multiplayer sessions and by validating edge cases in the shared game logic. In particular, turn order, round resets, score accumulation, wildcard handling, and draw-pile depletion were checked to ensure both players stayed synchronized. The shared test suite now also verifies that when the draw deck is exhausted, the accumulated discard pile is reshuffled while the current top discard card remains in play.

Result: PASS. No persistent synchronization faults were observed after the identified rules/packaging issues were corrected.

4.4 Maintainability

Maintainability was evaluated through the structure of the Rev 0 codebase and by adding repeatable automated checks. The system is separated into `frontend`, `backend`, and `shared` workspaces, allowing base conversion, rules, scoring, and game-engine logic to be tested in isolation. GitHub Actions now installs dependencies, builds all three workspaces, runs the shared unit tests, and archives a coverage report. This makes regressions easier to detect and reduces the risk of manual-only testing.

Result: PASS. The codebase is modular, typed, and now supported by automated build/test checks that improve confidence in future revisions.

—

5 Comparison to Existing Implementation

Traditional Crazy Eights implementations typically follow standard card rules without additional educational components. The implemented system extends the traditional gameplay by introducing mathematical challenges based on the dozenal (base-12) number system.

Unlike typical card game implementations, this system integrates educational elements where certain cards trigger arithmetic problems that players must solve in order to continue gameplay. This approach enhances both engagement and learning outcomes.

—

6 Unit Testing

To ensure the robustness of the The Crazy Tens, a total of 40 test cases were executed. The following table (continued across pages) document the verification of the mathematical core, game mechanics, and system security.

l p2.2cm X X l Unit Test Cases for Crazy 1-0s Game

ID	Component	Input / Action	Expected Output	Result

Table 0 – Continued from previous page

ID	Component	Input / Action	Expected Output	Result

Continued on next page

Category 1: Dozenal (Base-12) Math Engine

UT01 `logic.toDoz` Decimal: 10 String: "A" Pass
UT02 `logic.toDoz` Decimal: 11 String: "B" Pass
UT03 `logic.toDoz` Decimal: 12 String: "10" Pass
UT04 `logic.toDoz` Decimal: 144 String: "100" Pass
UT05 `logic.toDoz` Decimal: 1727 String: "BBB" Pass
UT06 `logic.fromDoz` String: "1A" Decimal: 22 Pass
UT07 `logic.fromDoz` String: "B1" Decimal: 133 Pass
UT08 `logic.val` String: "G" (Invalid) Throw `InputError` Pass

Category 2: Card Mechanics & Suit Triggers

UT09 `card.math` Play **Hearts (H)** Trigger Addition (+) Pass
UT10 `card.math` Play **Diamonds (D)** Trigger Subtraction (-) Pass
UT11 `card.math` Play **Clubs (C)** Trigger Multiplication (*) Pass
UT12 `card.math` Play **Spades (S)** Trigger Division (/) **FAIL**
UT13 `card.effect` Play **"Inverse 2"** Trigger Wildcard Menu **FAIL**
UT14 `card.effect` Play **"1-0"** Card Trigger "Crazy 1-0" Pass
UT15 `card.deck` Draw from 0 deck Reshuffle discard pile Pass
UT16 `card.suit` Wildcard: Select "Bells" Global Suit update Pass
UT17 `card.valid` Play "A" on "A" Accepted Pass
UT18 `card.valid` Play "B" on "4" Rejected Pass
UT19 `card.limit` Hand size > 20 Trigger Overflow penalty Pass
UT20 `card.draw` Player draws last card Event: "Deck Depleted" Pass

Category 3: Multiplayer & State Synchronization

UT21 `turn.lock` P2 plays during P1 turn Error: "Not your turn" Pass
UT22 `state.calc` Hand: [A, 5, 1-0] Total: 27 (Decimal) Pass
UT23 `socket.auth` Duplicate UID Join Reject Connection Pass
UT24 `socket.lag` 300ms latency spike Client-side prediction Pass
UT25 `state.race` Double Special Card Play Sequential State Lock **FAIL**
UT26 `lobby.full` 3rd player joins room Redirect to Full-Msg Pass
UT27 `socket.drop` Sudden DC P2 wins by default Pass
UT28 `state.sync` Mid-game page refresh Session restoration Pass
UT29 `turn.timer` Turn timeout (30s) Forced draw & Pass turn Pass
UT30 `msg.queue` Rapid 10x msg burst Server throttle check Pass

Category 4: UI, UX & Visuals

UT31	`ui.input`	Rapid-click "Draw" x5	Debounced: 1 draw	Pass
UT32	`ui.render`	Mobile Viewport (375px)	Flexbox Layout adjust	Pass
UT33	`ui.dozenal`	Toggle Base-12 mode	All labels update	Pass
UT34	`ui.anim`	Card Flip animation	Callback on Finish	Pass
UT35	`ui.modal`	Open Wildcard Menu	Background Blur active	Pass
Category 5: Security & Robustness				
UT36	`sec.inject`	Injection: "Score: 999"	Server side rejection	Pass
UT37	`sec.hand`	Play card NOT in hand	Log: "Illegal Move"	Pass
UT38	`input.san`	Chat: "<script>"	Escaped entities	Pass
UT39	`perf.mem`	1-hour session	Leak < 10MB increase	Pass
UT40	`perf.cpu`	Max 100 concurrent users	CPU load < 60%	Pass

6.1 In-depth Analysis of Critical Failures

The V&V process highlighted three failures that led to significant architecture improvements:

- **UT12 (Spades Trigger - Division):** Playing a Spades card triggered the division logic correctly. However, a "Division by Zero" bug was identified where the game engine crashed if a zero was generated as a divisor. We implemented a validator in the `ArithmeticModule` to prevent zero values.
- **UT13 (Inverse 2 Wildcard):** The "Inverse 2" failed to trigger the wildcard menu because the dispatcher only verified the card's rank (2) but not the `isInverse` boolean property. This was resolved by prioritizing property-based triggers.
- **UT25 (Race Condition):** Simultaneous special card plays caused overlapping React Modals and a UI deadlock. We introduced a `ModalMutex` to queue and synchronize these interactions via the server.

7 Changes Due to Testing

Testing, demo feedback, and a direct code review of Rev 0 led to the following gameplay and UI changes being recorded for the final rule set. Some

were implemented immediately in the codebase, while the larger arithmetic-interaction refinements were documented as approved follow-up work because they emerged directly from VnV activities.

- The starting hand size was standardized to 7 cards instead of 5, aligning the game setup with the intended Rev 0 rules and system-test expectations.
- The match goal was increased from 50 to 100 (displayed in the active numeral system), making multi-round play less abrupt.
- The wildcard rule was clarified so that rank `10` is the wildcard card; the earlier "Dek" / inverse-2 interpretation was removed from the accepted rule set.
- Score displays in Dozenal mode were normalized to avoid confusing or poorly rendered special glyphs in the UI.
- Draw-pile depletion behavior was re-checked in the shared rules engine: when the deck is exhausted, the accumulated discard pile is reshuffled back into the draw pile while the current top discard card remains on the table.
- The Decimal version of the game was confirmed to already exist in the Rev 0 implementation, and it is ready for demonstration in the next professor meeting.
- Demo feedback also showed that the lobby create/join experience was too complex, so the lobby flow was marked for simplification.
- Arithmetic interactions need a broader revision: answers should be typed instead of chosen from multiple-choice options; both players should see the same arithmetic prompt and race to answer; and the arithmetic ranges should be expanded to 100 for Dozenal play, with Decimal mode using 100 for addition/subtraction and 144 for multiplication/division.
- Client feedback further clarified that any face card should trigger an arithmetic challenge. In Decimal mode, `K` should let the player choose the operation; in Dozenal mode, `C` should provide that choice.

- Manual passing was deemed unnecessary for the intended experience; instead, move availability should be computed automatically from the player's hand before requiring further interaction.
- A single-round winner popup showing who won the round and how many points were earned was identified as a needed end-of-round improvement beyond the existing full-game end screen.

Feedback from early demonstrations also led to improvements in the user interface layout and clearer presentation of decimal versus dozenal values.

8 Automated Testing

Automated verification is now integrated into the development workflow using GitHub Actions. The repository currently has two relevant automation paths: one for the Rev 0 codebase and one for documentation builds.

The Rev 0 CI workflow (`.github/workflows/node.js.yml`) runs on pushes and pull requests that modify `src/rev0`. It performs the following:

- installs the workspace dependencies from `src/rev0/package-lock.json`
- builds the shared package
- runs the Vitest shared-logic unit test suite with V8 coverage enabled
- builds the backend
- builds the frontend
- uploads the generated shared coverage report as a GitHub Actions artifact

In addition, `.github/workflows/buildtex.yml` recompiles changed LaTeX documents on `main`, providing automated support for the report PDFs.

At this stage the project has meaningful **continuous integration (CI)** for code and documentation, but it does **not** yet have a full deployment pipeline for the application itself. Therefore, the current setup should be described as CI rather than complete CI/CD.

9 Trace to Requirements

Each functional requirement is verified by corresponding test activities.

Requirement	Test Type
FR-1	System Test
FR-2	Integration Test
FR-3	Unit Test
FR-4	System Test
FR-5	System Test
FR-6	Unit Test
FR-7	UI Test
FR-8	Manual Test
FR-9	UI Test

10 Trace to Modules

Module	Test Coverage
Game Engine	Unit + Integration
Rules Module	Unit Testing
Scoring Module	Unit Testing
Base Conversion Module	Unit Testing
UI Modules	Manual Testing

11 Code Coverage Metrics

Code coverage is now measured using Vitest with the V8 coverage provider. Because the current automated tests focus on the shared logic package (`src/rev0/shared`), the metrics below apply specifically to that package rather than to the entire repository. This avoids claiming a misleading repo-wide percentage before frontend and backend integration tests are automated.

Results from the current shared-package suite are:

- Overall statement coverage: 84.47%
- Overall branch coverage: 69.65%
- Overall function coverage: 98.07%
- Overall line coverage: 91.17%

File-level statement coverage for the shared package is summarized below:

File	Statement Coverage
`baseConversion.ts`	89.21%
`gameActions.ts`	100.00%
`gameEngine.ts`	83.01%
`rules.ts`	78.26%
`scoring.ts`	100.00%
`systems.ts`	100.00%

These results show that the critical shared logic is now exercised by repeatable automated tests, while also making it clear that branch coverage can still be improved in future work.

Appendix — Reflection

1. **What went well while writing this deliverable?** The strongest part of this deliverable was the modular structure of the Rev 0 codebase. Because the frontend, backend, and shared game logic are separated cleanly, it was practical to verify the core rule system, scoring, and base-conversion behavior directly and then connect those results back to the report. Converting placeholder testing claims into a real automated Vitest suite also made the final report more defensible.

2. **What pain points did you experience during this deliverable, and how did you resolve them?** The main pain point was distinguishing between report text that described intended behavior and behavior that was actually implemented in `src/rev0`. Resolving this required checking the code directly, building the project, and adding executable tests so that the VnV report could rely on measured evidence instead of assumptions.

3. **Which parts of this document stemmed from speaking to your client(s) or a proxy (e.g. your peers)? Which ones were not, and why?** The *Usability* evaluation and the larger portion of *Changes Due to Testing* came directly from client/proxy feedback, including the requests to simplify the lobby flow, clarify the wildcard rule, revise arithmetic interactions, and confirm the Decimal mode demo. By contrast, the *Automated Testing* and *Code Coverage* sections came from the team's internal verification work because those sections depend on build tooling, CI workflows, and measured coverage data rather than external feature feedback.

4. **In what ways was the Verification and Validation (VnV) Plan different from the activities that were actually conducted for VnV?** The original plan leaned more heavily on broad manual/system testing and assumed that the automated evidence would already exist. In practice, the final VnV work required adding a focused shared-package unit test suite, updating CI to run it automatically, and then using those measured results to support the report. The actual VnV activities therefore became a hybrid of manual UI validation, client/proxy feedback, and targeted automated testing of the shared game logic.